



عملي

4

9

1350 sp

كلية الهندسة المعلوماتية

السنة الرابعة

BFS & DFS With UCS

عملي مشترك



RB Informatcs; 19/11/2020

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات البحث الذكية

مقدمة:

تحدثنا سابقاً عن طريقة بناء اللعبة الخاصة بنا واليوم سنتحدث عن أولى خوارزميات البحث الـ BFS و الـ DFS سبق وذكرناهما في المحاضرات السابقة واليوم سنتطرق لشرحهما بالتفصيل مع إيجابيات وسلبيات كل منهما مع طرق تحسينهما.

BFS (Breadth-First Search)

ما تفعله هذه الخوارزمية ببساطة هو استكشاف جميع الجيران في العمق الحالي قبل الذهاب الى العمق الأعظم، يمكن تشبيهه بفيضان المياه ينتشر من نقطة البداية بالتساوي في جميع الاتجاهات.

خصائصها:

• استخدام قائمة الانتظار. Queue

• يضمن أقصر مسار في الـ graph غير الموزون (أقل عدد خطوات).

• يستكشف العقد طبقة تلو أخرى.

متى تستخدم هذه الخوارزمية :

• إيجاد أقصر طريق في متاهة أو شبكة في حال تساوي الـ Cost.

• ألغاز مثل الـ Sliding puzzle ,Rubik's cube أو خريطة كنز.

مثال من لعبة:

تخيل اللاعب يقف في الإحداثيات (2, 3) والهدف (goal) في الإحداثيات (0, 5)

واللاعب يستطيع الحركة D, U, L, R

عن طريق خوارزمتنا (BFS) سيصل للهدف بـ 4 خطوات فقط بحيث تكون أقصر خوارزمية من حيث عدد الخطوات.

3	2	1	2	3
2	1		1	2
3	2	1	2	3
	3	2	3	

DFS(Depth-First Search)

ما تفعله هذه الخوارزمية هو ببساطة هو الذهاب لأعمق عمق ممكن في كل فرع وعندها يعود للخلف خطوة ثم في غير الفرع التالي.

خصائصها:

- يستخدم المكس (Stack) أو ال (recursion) العودية (LIFO).
- لا يضمن أقصر حل.
- يدخل للعمق بسرعة كبيرة.
- يستخدم ذاكرة أقل من ال BFS في بعض الحالات (خصوصاً مع العودية).

متى تستخدم؟

- التحقق من وجود طريق وجود طريق أساساً بين البداية والهدف.
- إيجاد كل الطرق الممكنة أو سردها.
- ألعاب ال backtracking مثل حل سودوكو، متاهة بحلول متعددة، توليد متاهة.
- بترتيب طوبولوجي كشف الحلقات في ال graph.

جدول مقارنة بين DFS & BFS

الخاصية	BFS	DFS
هيكل البيانات	Queue	Stack Recursion
الاستكشاف	طبقة بطبقة	العمق أولاً
أقصر طريق	نعم	لا
استهلاك الذاكرة	أعلى	عادة أقل
السرعة	أبطأ عادة	أسرع

أمثلة حقيقية مساعدة:

استكشاف زلزلة (DFS) , صيد الكنوز (BFS) , Snake(BFS) , pac-Man (DFS)

AI with path Finding
 BFS (العدو يتحرك بأذكى طريقة نحو اللاعب) + DFS (العدو يستكشف المنطقة أولاً)
 8 Puzzles
 BFS (حل بأقل عدد خطوات) + DFS (حل مقبول مع تحديد العمق IDDFS)
 حيلة سهلة لحفظ الخوارزميتين :
 BFS = Best For Shortest path
 DFS = Dive First See What happens

مساوئ كل خوارزمية (DFS, BFS) في سياق الألعاب والبحث:

الخوارزمية	مساوئ	أمثلة في الألعاب
BFS	- استهلاك ذاكرة كبير جداً. - بطيء في ال graph العميق. - غير مناسبة لل graph ذات اوزان مختلفة. - يحتاج إلى زيارة كل عقدة في الطبقة قبل الانتقال.	- في المتاهة الكبيرة او خريطة العال المفتوح - قد يملأ ال queue ذاكرة كبيرة جدا قبل إيجاد الحل
DFS	- أحيانا لا يعطي - قد يعلق في مسارات طويلة جداً. - مشكلة في ال graph ذات ال Cycles. - StackOverflow .	كما في العاب المتاهات قد يذهب ال dfs الى طريق عميق جدا قبل ان يرجع مما يجعل العدو غبي او بطئ

نصائح لتحسين الأداء في الألعاب والبحث :

DFS	BFS
- Iterative Deepening Depth-First Search (IDDFS) يستخدم ذاكرة قليلة مع حد عمق متزايد على DFS - VisitedSet لتجنب الدورات - Pruning with backtracking إضافة شروط لقطع الفروع الغير مفيدة مبكراً. - استخدام recursion بدل Stack	- استخدام هياكل بيانات فعالة ففي البايثون استخدام ال deque بدل queue وفي الجافا يمكن استخدام linked list بدل arraylist - استخدام Visited set - قص الفرع المبكر (Early Termination) - Bidirectional BFS وهي خوارزمية تشبه BFS لكنها تبدأ من البداية وبنفس الوقت من الهدف بالوقت نفسة (يلتقيان في الوسط) تقلل عدد العقد المزاراة بشكل كبير

دالة BFS (نقطة_البداية, الهدف , ال graph او الشبكة):

أنشئ قائمة انتظار queue, أضف نقطة_البداية إلى queue, أنشئ مجموعة visited (للعقد المزاراة) , أضف نقطة_البداية إلى visited

أنشئ قاموس parent (لحفظ الطريق لإعادة بنائه لاحقاً): parent[نقطة_البداية] = null

بينما queue ليست فارغة: current = أزل العنصر الأول من queue (dequeue)

إذا current == الهدف

أعد بناء الطريق باستخدام parent ارجع الطريق
 لكل جار neighbor من current في الجراف/الشبكة:
 إذا neighbor غير موجود في visited:
 أضف neighbor إلى queue
 أضف neighbor إلى visited
 parent[neighbor] = current
 // إذا وصلنا هنا يعني ما في طريق ارجع "لا يوجد طريق"

دالة DFS(current, الهدف, parent, visited):

أضف current إلى visited
 parent[current] = الوالد الحالي (أو null إذا بداية)
 إذا current == الهدف :
 أعد بناء الطريق باستخدام parent ارجع الطريق
 لكل جار neighbor من current في الغراف / الشبكة:
 إذا neighbor غير موجود في ال visited :
 نتيجة = DFS(neighbor, الهدف, parent, visited)
 إذا نتيجة != "لا يوجد طريق": ارجع نتيجة
 ارجع "لا يوجد طريق" // طريق مسدود
 // استدعاء الدالة الرئيسية:
 دالة ابدأ DFS(نقطة_البداية, الهدف):
 أنشئ مجموعة visited فارغة
 أنشئ قاموس parent
 parent[نقطة_البداية] = null
 ارجع DFS(نقطة_البداية, الهدف, parent, visited)

Iterative(نقطة_البداية, الهدف, الجراف):

أنشئ مكس stack
 أضع نقطة_البداية إلى stack
 أنشئ visited و parent
 أضف نقطة_البداية إلى visited
 parent[نقطة_البداية] = null
 بينما stack ليست فارغة:

current = أزل العنصر الأعلى من stack (pop)

إذا current == الهدف:

أعد بناء الطريق

ارجع الطريق

لكل جار neighbor من current:

إذا neighbor غير موجود في visited:

أضف neighbor إلى visited

parent[neighbor] = current

أدفع neighbor إلى stack

ارجع "لا يوجد طريق"

دالة بناء_الطريق(parent, الهدف):

طريق = قائمة فارغة

current = الهدف

بينما current != null:

أضف current في بداية الطريق

current = parent[current]

ارجع الطريق

(Uniform Cost Search - UCS)

تخيلوا أنكم تخططون رحلة بالسيارة من مدينة إلى أخرى، والطرق بين المدن لها مسافات مختلفة: بعضها قصير، وبعضها طويل، وبعضها طرق سريعة وبعضها مليء بالمنعطفات. الهدف ليس الوصول بأقل عدد من الطرق، بل الوصول بأقل مسافة كلية (أو أقل تكلفة). هنا تأتي خوارزمية البحث بالتكلفة المنتظمة (UCS) لتحل هذه المشكلة بطريقة ذكية وعادلة.

UCS هي خوارزمية بحث في الجرافات تهدف إلى إيجاد الطريق ذي التكلفة الإجمالية الأقل من نقطة البداية إلى الهدف. تعتبر امتداداً طبيعياً لخوارزمية BFS، لكن مع فارق جوهري: بدلاً من توسيع العقد طبقة بطبقة (حسب عدد الخطوات)، توسع UCS العقد حسب التكلفة التراكمية الأقل حتى الآن.

المبادئ الأساسية للخوارزمية:

- تستخدم طابور أولويات (Priority Queue) يرتب العقد حسب التكلفة التراكمية من البداية (تسمى $g(node)$).
- في كل خطوة، تخرج العقدة ذات التكلفة الأقل من الطابور وتوسع جيرانها.
- إذا وجدت طريقاً أرخص إلى عقدة سبق زيارتها، تقوم بتحديث التكلفة وتعيد إدراجها في الطابور.

- تستمر حتى تصل إلى الهدف، وتكون النتيجة مضمونة أنها الطريق الأمثل (أقل تكلفة) طالما أن جميع التكاليف إيجابية.

لماذا تُستخدم UCS في الألعاب والتطبيقات؟

- في الألعاب مثل ألعاب المقاتلات أو الخرائط المفتوحة (Open World)، قد تكون تكلفة التحرك في العشب أعلى من التحرك على الطريق المعبّد، أو قد تكون بعض الحركات تستهلك طاقة أكثر.
- في تطبيقات الملاحة (مثل خرائط جوجل)، تمثل التكلفة المسافة أو الوقت المتوقع.
- في ألعاب الاستراتيجية أو الذكاء الاصطناعي للأعداء، تساعد UCS في جعل حركة الشخصيات واقعية وذكية.

مقارنة سريعة مع BFS و DFS :

BFS : يجد أقصر طريق بعدد الخطوات (عندما تكون التكاليف متساوية).

DFS : يجد أي طريق بسرعة، لكنه لا يهتم بالتكلفة.

UCS: يجد أقصر طريق من حيث التكلفة الكلية (حتى لو كانت التكاليف مختلفة).

المزايا	العيوب
تضمن الحل الأمثل. (optimal)	قد تكون بطيئة في الخرائط الكبيرة إذا لم تحسّن.
كاملة (complete) إذا كانت التكاليف إيجابية وبحد أدنى.	تستهلك ذاكرة كبيرة مثل (BFS) لأنها تحتفظ بكل المسارات المحتملة في الطابور.
مرنة لأي نوع من التكاليف الإيجابية.	

نصيحة لكم :

إذا كانت التكاليف متساوية، فإن UCS تتصرف تماماً مثل BFS، لكن BFS أسرع لأنه يستخدم طابوراً عادياً. أما إذا أردتم تحسيناً أكبر، فانتقلوا لاحقاً إلى خوارزمية A* التي تضيف تقديراً (heuristic) لتسريع البحث.

دالة UCS (نقطة_البداية، الهدف، الجراف):

أنشئ طابور أولويات priority_queue مرتب حسب التكلفة التراكمية و
أدرج (نقطة_البداية، تكلفة = 0) في priority_queue
أنشئ قاموس cost_so_far // أقل تكلفة معروفة للوصول إلى كل عقدة
cost_so_far[نقطة_البداية] ← 0
أنشئ قاموس parent // لإعادة بناء الطريق
parent[نقطة_البداية] ← لا شيء

بينما priority_queue ليست فارغة:

أخرج العنصر ذا التكلفة الأقل: current، current_cost

إذا current == الهدف:

أعد بناء الطريق باستخدام parent

ارجع الطريق والتكلفة الإجمالية

لكل جار neighbor من current في الجراف:

$\text{current_cost} \leftarrow \text{new_cost} + \text{تكلفة_الحافة}(\text{neighbor}, \text{current})$

إذا neighbor غير موجود في cost_so_far أو $\text{new_cost} < \text{cost_so_far}[\text{neighbor}]$:

$\text{cost_so_far}[\text{neighbor}] \leftarrow \text{new_cost}$

$\text{parent}[\text{neighbor}] \leftarrow \text{current}$

أدرج (new_cost, neighbor) في priority_queue

ارجع "لا يوجد طريق"

لعبة اربسيزية :

لعبتنا اليوم math push

هي لعبة ألغاز ممتعة على طراز **Sokoban** (دفع الكتل)، متوفرة مجاناً على CrazyGames.

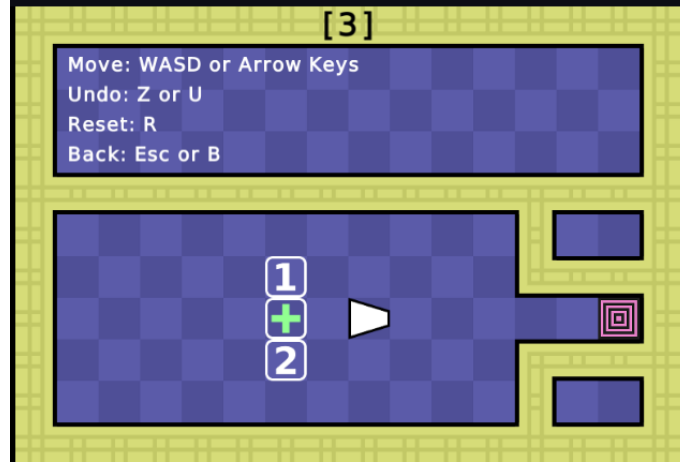


- الهدف: قم بدفع كتل تحمل أرقاماً (1-9) وعمليات رياضية (+, -) لتشكيل معادلات رياضية تطابق أرقاماً محددة على أبواب/أقفال تسد الخروج (البوابة). مثال: دفع كتل لتشكيل $3 = 2 + 1$ أو $7 = 5 - 12$ (يمكن دمج رقمين لصنع رقمين أرقام).

- كيفية اللعب: تحكم في سهم أبيض (اللاعب) باستخدام **WASD** أو الأسهم **لدفع الكتل دون الوقوع في مأزق. اضغط **R** لإعادة البدء، **U** للتراجع، **ESC** للقائمة.

- المستويات: تبدأ بسيطة وتزداد صعوبة، تتطلب تخطيطاً لتجنب حبس الكتل.

- القيمة التعليمية: تجمع الرياضيات البسيطة (جمع، طرح) مع المنطق والتخطيط – مثالية لتطبيق خوارزميات البحث! يمكنكم تجربة اللعبة من الرابط التالي : <https://www.crazygames.com/game/math-push>



مسابقة نهاية المحاضرة: "حل Math Push باستخدام UCS، DFS، BFS!"

الفكرة العامة: نموذج حالة اللعبة ك جراف حالات (State Graph):

- الحالة: موقع اللاعب + مواقع كل الكتل (مصفوفة D2 أو tuple).
- الإجراءات: دفع كتلة في 4 اتجاهات (إذا ممكن).
- الهدف: تشكيل المعادلات الصحيحة + وصول اللاعب للبوابة.
- التكلفة (UCS): افترض تكلفة 1 لكل دفع (أو زد للدفعات الطويلة).
- فرق الطلاب (3-5 أشخاص): نفذوا solver لمستوى بسيط (مثل المستوى 1-3، صمّموه أو استخدموا صورة من اللعبة).
- قارنوا النتائج: أقصر حل (BFS)، أي حل (DFS)، أقل تكلفة (UCS).

التحديات الثلاثة:

1. BFS Challenge (أقصر تسلسل حركات):

- ابحث عن أقل عدد دفعات لحل المستوى.
- الفائز: أسرع تنفيذ + أقصر حل.

2. DFS Challenge (أي حل سريع):

- ابحث عن أي حل صالح (مع backtracking).
- الفائز: أسرع اكتشاف حل + تجنب stack overflow.

3. UCS Challenge (أمثل تكلفة):

- أضف تكاليف (دفع كتلة بعيدة = تكلفة أعلى، أو حسب مسافة).
- الفائز: أقل تكلفة إجمالية + مقارنة مع BFS.
- الجوائز: أفضل عرض (5 دقائق) + نقاط إضافية. وقت: 20-30 دقيقة تنفيذ + 10 عرض.

↩ أسئلة مساعدة للطلاب (Helper Questions)

لتبسيط التنفيذ، أجبوا على هذه قبل الكود:

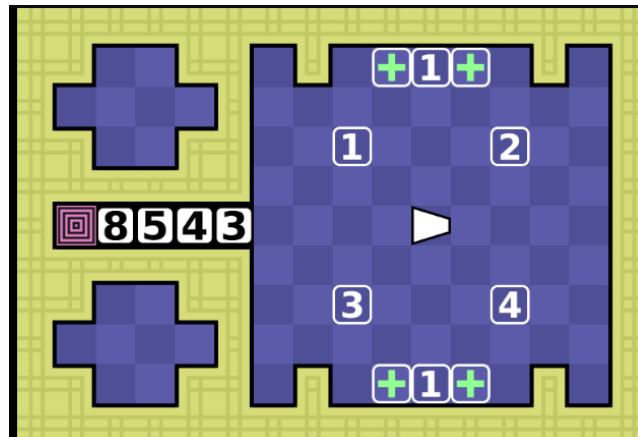
عامة (لثلاثة):

1. كيف تمثلون حالة اللعبة؟ (مثال: tuple من مواقع الكتل + موقع اللاعب).
2. ما هي الإجراءات الممكنة من حالة؟ (دفع في 4 اتجاهات، تحقق عدم اصطدام).
3. كيف تتحققون من الهدف؟ (تحليل الصفوف: جمع/طرح = رقم الباب؟).
4. كيف تمنعون التكرار؟ (visited set للحالات الهاش).

خاصة بكل خوارزمية:

- BFS: كيف تضمنون أقصر مسار؟ (Queue + visited).
- DFS: كيف تتعاملون مع العمق الكبير؟ (Recursion أم Stack؟ حد عمق؟).
- UCS: ما هي دالة التكلفة؟ (و = عدد دفعات، أم مسافة؟ Priority Queue كيف؟).

نصيحة: ابدأوا بمستوى بسيط، استخدموا deque Python + queue، PriorityJ heapq. شاركوا الكود على GitHub للعرض.... جربوا اللعبة أولاً! 😊
إذا استطاعت خوارزمياتك الثلاث حل المرحلة 5 فأنت صاحب IQ عالي



The End ...